This Document has been modified with Flexcil app (Android) https://www.flexcil.com

Algo 1

```
int sum = 0;
for (i=0; i<n; i++) {
    sum += i;
}
```

$O(n)$

2

```
int sum = 0;
for (i=0; i<n; i+=2) {
    sum += i
}
```

$O(\frac{n}{2}) \longrightarrow$ Drop constant!

$= O(n)$

3

```
for (i=0; i<n; i *= 2) {
    sout(i)
}
```

$O(\log(n))$

4

```
for (i=0; i<n; i++) {
    for (j=0; j<n; j++) {
        sout(i+j);
    }
}
```

$O(n^2)$

5 Dependant loop

```
for (i=0; i<n; n++) {
    for (j=0, j<i; j++) {
        sout(i+j)
    }
}
```

$O(n^2)$

Flexcil - The Smart Study Toolkit & PDF, Annotate, Note

This Document has been modified with Flexcil app (Android) https://www.flexcil.com

6

```
for (i = 1, i < n; i *= 2) {
  for (j = 0, j < n; j++) {
    Sout(i + j)
  }
}
```

Outer loop = $O(\log n)$

inner loop = $O(n)$

$O(n \cdot \log(n))$

7

```
for (i = n; i > 0; i /= 2) {
    for (j = n; j > 0; j /= 2) {
        for (k = 0; k < n; k++) {
            Sout(i + j + k);
        }
    }
}
```

i) $\begin{cases} O(\log(n)) \\ O(\log(n)) \\ O(n) \end{cases} = O(n(\log n)^2)$

8

```
Void recursive(int n) {
    if (n <= 1) return;      Base Case
    recursive(n/2);          Recursive call
    recursive(n/2);          Recursive call
}
```

$T(n) = 2T(n/2) + O(1)$

Base case → $f(n)$

rec(n) — 1 call

rec(n/2) rec(n/2) — 2 calls

rec(n/2) rec(n/2) rec(n/2) rec(n/2) — 4 calls

8 calls

level k has 2^k calls

Find Height of tree

$\frac{n}{2^k} = 1$

$2^k = n$

$k = \log_2 n$

$2^{\log_2 n} = n$

$O(n)$

Flexcil - The Smart Study Toolkit & PDF, Annotate, Note

This Document has been modified with Flexcil app (Android) https://www.flexcil.com

Solve with Induction

$T(n) = 2T(n/2) + 1$

$T(n/2) = 2T(n/4) + 1$

$T(n/4) = 2T(n/8) + 1$

$T(n) = 2(2T(n/4) + 1) + 1$

$= 4(T(n/4) + 3$

$T(n) = 8(T(n/8) + 7$

$T_n = 2^k(T(n/2^k) + (2^k - 1))$

$n/2^k <= 1$ Base case

$2^k = n$

$T_n = (n \cdot T(1) + (n-1)$

$\Rightarrow T_n = 2n - 1$

$O(n)$

```
Void recursive(int n) {
    if (n <= 1) return;      Base Case
    recursive(n/2);          Recursive call
    recursive(n/2);          Recursive call
    recursive(n/2);
}
```

$T(n) = 3T(n/2) + 1$

$T(n/2) = 3T(n/4) + 1$

$T(n/4) = 3T(n/8) + 1$

$\Rightarrow T(n) = 9(T(n/4) + 4$ 2

$= 27(T(n/8) + 13$ 3

$T(n) = 3^k(T(n/2^k) + (3^k - 1)/2$

$2^k/n = 1$

$2^k = n$

$k = \log_2 n$

$T_n = 3^{\log_2 n} = n^{\log_2 3}$

$n^{\log_3} \cdot T(1) + (n^{\log 3} - 1)/2$

$O(n^{\log_2 3})$

Flexcil - The Smart Study Toolkit & PDF, Annotate, Note

This Document has been modified with Flexcil app (Android) https://www.flexcil.com


```
Void recursive (int n) {
    if (n <= 1) return;
    recursive(n-1);
    recursive(n-1);
}
```

$T(n) = 2T(n-1) + 1$

$= 2(2T(n-2) + 1) + 1$

$= 4T(n-2) + 3$

$= 4(2T(n-3) + 1) + 3$

$= 8T(n-3) + 7$

$T(n) = 2^k T(n-k) + 2^k - 1$

$n - k = 1$

$\Rightarrow k = n - 1$

$T(n) = 2^{n-1} T(1) + (2^{n-1} - 1)$

$= 2^{n-1} + 2^{n-1} - 1$

$= 2^n - 1$

$T(n) = O(2^n)$

Flexcil - The Smart Study Toolkit & PDF, Annotate, Note